

Day 14

Big Data, Distributed Systems & Apache Spark / Hadoop

Cloud Computing & Big Data Analytics — Digital Science Programme

Cyber.SoHo Educational Hub | © 2026

Agenda — Today's Journey

- 1. What Is Big Data? — The Five V's Framework
- 2. Why Traditional Systems Fail at Scale
- 3. Distributed Systems — Core Concepts & CAP Theorem
- 4. Apache Hadoop — Architecture & Ecosystem
- 5. HDFS & MapReduce — Deep Dive
- 6. Apache Spark — The In-Memory Engine
- 7. Spark vs. Hadoop — When to Use What
- 8. Real-World Big Data Pipelines in Industry

LESSON 1

What Is Big Data?
The Five V's Framework

What Is Big Data?

- Not just "a lot of data" — a specific class of datasets
- So massive, varied, and fast that conventional databases cannot handle them
- Classified using the Five V's Framework
- Originally 3 V's by Doug Laney at Gartner (global research firm)
- Expanded to 5 V's by the data science community
- Key unit scales: GB → TB → PB (Petabyte) → EB (Exabyte) → ZB (Zettabyte)

The Five V's — Volume & Velocity

- **VOLUME — How Much Data**
- Measured in PB (Petabytes) and EB (Exabytes)
- YouTube: 500+ hours of video uploaded every single minute
- 1 PB $\approx$ 13.3 years of HD video

- **VELOCITY — How Fast It Arrives**
- Data streams continuously, often in real time
- Stock markets: millions of transactions per second
- IoT (Internet of Things) sensors: continuous telemetry

The Five V's — Variety & Veracity

- **VARIETY — What Formats**
 - **Structured:** Spreadsheets, SQL (Structured Query Language) tables
 - **Semi-structured:** JSON (JavaScript Object Notation), XML logs, CSV files
 - **Unstructured:** Images, videos, emails, social media posts
- **VERACITY — How Trustworthy**
 - **Noise, duplicates, and missing values are extremely common**
 - **Must filter bots, sarcasm, and spam from genuine signals**

The Five V's — Value

- **VALUE — What Business Insight It Yields**
- Raw data is worthless without processing and interpretation
- Netflix saves ~\$1B USD/year by analysing viewing data to reduce churn
- Hospitals redesign treatment protocols from outcome analysis
- The Value V is the ultimate goal — all other V's exist to serve it

★ **Key Takeaway:** Volume, Velocity, Variety, Veracity, Value — the standard Big Data classification framework.

LESSON 2

Why Traditional Systems
Crack Under Pressure

Vertical Scaling — "Scale Up"

- Buy a bigger, faster, more expensive single server
- **Limitations:**
 - Hardware ceiling — physical cap on RAM and CPU cores
 - Cost explosion — doubling capacity can triple the price
 - SPOF (Single Point of Failure) — one server down = total outage
 - I/O (Input/Output) bottleneck — finite disk & network throughput

Horizontal Scaling — "Scale Out"

- Many smaller, commodity machines networked together
- Each machine handles a portion of the workload in parallel
- If one fails, the others keep running
- **Kitchen Analogy:**
- Vertical = one enormous stove + one superhuman chef
- Horizontal = 50 kitchen stations + 50 cooks working in parallel
- If one cook calls in sick, the other 49 keep going

★ **Key Takeaway:** Horizontal scaling underpins all modern Big Data tech — many cheap machines beat one expensive one.

LESSON 3

Distributed Systems
The Brain Behind the Scale

What Is a Distributed System?

- A collection of independent computers appearing as a single system
- Nodes communicate over a network and coordinate actions
- Powers everything: Google Search, Netflix, banking, ride-sharing
- **Pioneers:**
 - Leslie Lamport — Turing Award, created LaTeX, foundational theory
 - Eric Brewer — CAP Theorem, UC (University of California) Berkeley
 - Andrew Tanenbaum — textbooks that trained generations of engineers

The CAP Theorem — Pick Two of Three

Property	Meaning	Example
C — Consistency	Every read returns the most recent write	Bank balance always current
A — Availability	Every request gets a response	System never says "unavailable"
P — Partition Tolerance	System works despite network splits	Both data centres keep running

★ **Key Takeaway:** Network partitions WILL happen — you choose CP (consistent) or AP (available), never all three.

Core Distributed Systems Concepts

- **Replication**
 - Copies of data on multiple nodes for fault tolerance
- **Sharding (Partitioning)**
 - Splitting datasets across nodes — each holds a subset
- **Consensus Algorithms (Paxos, Raft)**
 - Protocols for nodes to agree on values despite failures
- **Fault Tolerance**
 - System keeps running when components break
- **Latency**
 - Time delay between request and response

LESSON 4

Apache Hadoop
The Elephant That Changed Everything

Hadoop — Origin Story

- **2003: Google publishes GFS (Google File System) paper**
- **2004: Google publishes MapReduce paper**
- **Doug Cutting (Yahoo!) builds open-source implementation**
- **Named Hadoop after his son's toy stuffed elephant**
- **Donated to Apache Software Foundation (ASF)**
- **Became the cornerstone of the entire Big Data revolution**
- **Key insight: cheap commodity servers > expensive supercomputers**

Hadoop Core Components

Component	What It Does
HDFS (Hadoop Distributed File System)	Stores files in 128 MB blocks; replicates each block 3×
YARN (Yet Another Resource Negotiator)	Manages CPU/memory; schedules jobs across the cluster
MapReduce	Programming model: Map (transform) → Reduce (aggregate)
Hadoop Common	Shared Java libraries and utilities for the ecosystem

The Hadoop Ecosystem – Beyond Core

- **Apache Hive**
 - SQL-like queries on Hadoop data
- **Apache HBase**
 - NoSQL (Not Only SQL) column-family database on HDFS
- **Apache Kafka**
 - Distributed event streaming for data ingestion
- **Apache Sqoop / Flume / ZooKeeper**
 - Data transfer, log ingestion, and coordination services

★ **Key Takeaway:** Hadoop is an ecosystem, not a single tool. HDFS stores, YARN schedules, MapReduce processes.

LESSON 5

HDFS & MapReduce
Demystified

HDFS Architecture — Master/Worker

- **NameNode (Master)**
 - Maintains file system namespace and block-to-node metadata map
 - Does NOT store actual data — only the directory tree
- **DataNodes (Workers)**
 - Store actual data blocks (128 MB default)
 - Send periodic heartbeat signals to NameNode
- **Replication Factor = 3**
 - Each block stored on 3 different DataNodes for fault tolerance

MapReduce – The Two-Phase Model

- **Phase 1: MAP**
 - Each node processes its local data block
 - Produces intermediate key-value pairs (tagging data)
- **Phase 2: SHUFFLE & SORT**
 - Framework groups all pairs by key across nodes
- **Phase 3: REDUCE**
 - Takes each group of same-key pairs
 - Aggregates into final result (sum, count, average)

MapReduce Example – Word Count

- **INPUT:**
 - "the cat sat" | "the dog sat" | "the cat ran"
- MAP → (the,1)(cat,1)(sat,1)(the,1)(dog,1)(sat,1)(the,1)(cat,1)(ran,1)
- SHUFFLE → the:[1,1,1] | cat:[1,1] | sat:[1,1] | dog:[1] | ran:[1]
- REDUCE → the=3, cat=2, sat=2, dog=1, ran=1

★ **Key Takeaway:** MapReduce decomposes analytics into Map (transform) + Reduce (aggregate), running in parallel.

LESSON 6

Apache Spark
The Lightning-Fast Successor

Why Spark Was Created

- **MapReduce's Fatal Flaw:**
 - Every step writes output TO DISK before the next step reads it
 - For iterative algorithms (ML with 100+ passes) → devastating overhead
- **Spark's Innovation (Matei Zaharia, UC Berkeley, 2009):**
 - Introduced RDD (Resilient Distributed Dataset)
 - Immutable data collections living IN MEMORY across the cluster
 - Up to 100× faster than disk-based MapReduce for iterative workloads
 - Donated to Apache Software Foundation in 2013

Spark Architecture — Components

Component	What It Does
Spark Core	Task scheduling, memory mgmt, fault recovery, RDD API
Spark SQL	Query structured data using SQL and DataFrames
Spark Streaming	Process real-time data streams (IoT, live feeds)
MLlib (Machine Learning Library)	Scalable classification, regression, clustering
GraphX	Graph processing (social networks, page ranking)

How Spark Runs a Job

- **1. Driver Program**
 - Your Python / Scala / Java code defining the computation
- **2. SparkContext**
 - Entry point connecting the driver to the cluster
- **3. Cluster Manager**
 - Allocates resources — YARN, Kubernetes, or Standalone
- **4. Executors**
 - Worker processes on nodes — run tasks, store data in RAM

Transformations vs. Actions — Lazy Evaluation

- **Spark uses LAZY EVALUATION — nothing happens until an Action**
- **Transformations (Lazy — define the plan):**
 - `map()`, `filter()`, `flatMap()`, `groupByKey()`
- **Actions (Eager — trigger execution):**
 - `collect()`, `count()`, `saveAsTextFile()`, `reduce()`
- **Transformations build a DAG (Directed Acyclic Graph)**
- **Actions trigger the DAG execution and produce output**

★ **Key Takeaway:** Spark keeps data in RAM + uses lazy evaluation via DAG — transformations plan, actions execute.

LESSON 7

Spark vs. Hadoop
Knowing When to Use What

Spark vs. Hadoop — Speed & Processing

Criterion	Hadoop MapReduce	Apache Spark
Processing	Disk-based	In-memory
Speed	Slower for iterative work	Up to 100× faster for ML
Ease of use	Verbose Java code	Clean Python/Scala APIs
Streaming	Requires add-ons	Built-in Spark Streaming

Spark vs. Hadoop – Fault Tolerance & Cost

Criterion	Hadoop MapReduce	Apache Spark
Fault tolerance	Replicates data blocks 3×	Tracks RDD lineage; recomputes
Cost	Cheaper for cold-storage batch	Needs more RAM (pricier)
ML Support	External (Mahout)	Built-in MLlib
Best for	Massive batch ETL	Iterative ML, streaming, SQL

When to Use Hadoop

- **Petabytes of archival data processed rarely**
- **Budget prioritises disk storage cost over processing speed**
- **Batch ETL (Extract, Transform, Load) on cold data**
- **Regulated industries (banking, government) needing proven reliability**
- **Long-term cold storage with infrequent access patterns**

When to Use Spark

- **Fast, iterative processing (ML training with 100+ epochs)**
- **Real-time / near-real-time analytics on streaming data**
- **Developer-friendly Python (PySpark) or Scala APIs**
- **Need a unified engine for batch + streaming + SQL + ML**
- **Interactive data exploration and ad-hoc queries**

The Modern Reality – They Work Together

- Many organisations run Spark ON TOP OF Hadoop's YARN and HDFS
- Hadoop provides storage + resource management layer
- Spark provides fast in-memory processing engine
- Not either/or — a complementary layered architecture
- **Common Stack:**
 - HDFS (storage) + YARN (scheduling) + Spark (processing)

★ **Key Takeaway:** Hadoop stores + schedules. Spark processes. Together they form a powerful production stack.

LESSON 8

Real-World Big Data
Pipelines in Industry

Who Uses Big Data – And How

Company	Stack	Use Case
Netflix	Spark + Kafka + Cassandra	500B events/day for recommendations
Uber	Spark + Hadoop + Hive	Surge pricing, ETA prediction
Spotify	Spark + GCP BigQuery	Discover Weekly from billions of listens
NASA	Hadoop + Spark	Satellite imagery & mission telemetry
CERN	Spark + Hadoop + ROOT	Petabytes of particle collision data

Anatomy of a Big Data Pipeline

- **Data Sources → Ingestion → Storage → Processing → Analysis → Visualisation**
- **Data Sources:**
 - IoT sensors, application logs, APIs, databases
- **Ingestion:**
 - Apache Kafka, Apache Flume (real-time streaming)
- **Storage:**
 - HDFS, Amazon S3, Azure Blob Storage
- **Processing:**
 - Apache Spark, MapReduce (batch + stream)
- **Analysis + Visualisation:**
 - SparkSQL, MLlib → Tableau, Grafana, Power BI

Career Paths in Big Data

- **Data Engineer**
 - Builds and maintains pipelines (Hadoop, Spark, Kafka, Airflow)
- **Data Scientist**
 - PySpark, MLlib, and statistical models for insights
- **Data Analyst**
 - Queries processed data via SparkSQL, Hive, BigQuery
- **ML (Machine Learning) Engineer**
 - Trains and deploys models at scale (Spark MLlib, distributed TF)
- **Platform Engineer / SRE**
 - Keeps Hadoop/Spark clusters healthy and performant

★ **Key Takeaway:** Data Engineers build it, Data Scientists analyse it, ML Engineers scale it.

Key Technologies – Quick Reference

Technology	Type	Primary Use
HDFS	Storage	Distributed file storage across cluster
YARN	Resource Mgmt	CPU/memory allocation, job scheduling
MapReduce	Processing	Batch transform and aggregate
Apache Spark	Processing	In-memory batch, streaming, ML, SQL
Apache Kafka	Ingestion	Real-time event streaming
Apache Hive	Query	SQL interface on Hadoop data
HBase	Database	NoSQL column-family store on HDFS
ZooKeeper	Coordination	Distributed consensus & config

The Five V's – Quick Recall

V	One-Liner
Volume	How much data (petabytes+)
Velocity	How fast it arrives (real-time streams)
Variety	What formats (structured, semi, unstructured)
Veracity	How trustworthy (noise, errors, bias)
Value	What business insight it yields

Summary — Eight Lessons in Eight Lines

- 1. Big Data = Five V's: Volume, Velocity, Variety, Veracity, Value
- 2. Vertical scaling hits ceilings; horizontal scaling breaks through
- 3. Distributed systems trade CAP properties for resilience at scale
- 4. Hadoop democratised Google's infrastructure for the world
- 5. HDFS splits and replicates; MapReduce transforms and aggregates
- 6. Spark keeps data in memory — up to 100× faster for iterative jobs
- 7. Hadoop and Spark complement each other in modern architectures
- 8. Big Data powers Netflix, Uber, Spotify, NASA, and CERN daily

What's Next — Day 15 Preview

- **Hands-on lab: Setting up a local Spark cluster**
- **PySpark DataFrames and Spark SQL**
- **Writing your first distributed word-count job**
- **Connecting Spark to cloud storage (S3, Azure Blob, GCS)**

Thank You

Day 14 — Big Data, Distributed Systems & Apache Spark / Hadoop

Cloud Computing & Big Data Analytics
Digital Science Programme

Cyber.SoHo Educational Hub | © 2026
Questions? Discussion? Let's go!